Requisitos Funcionais (RF)

Já Implementado:

- **[RF-01]** O sistema deve emitir e validar tokens JWT para cadastro e autenticação de usuários.
- **[RF-02]** O sistema deve separar as permissões (RBAC) com base em dois perfis de acesso: USER (Turista) e ADMIN/GUIA.
- **[RF-03]** O sistema deve permitir que o administrador cadastre Pontos Turísticos no banco de dados.
- **[RF-04]** O sistema deve possuir mapas restritos ao limite geográfico do Rio Grande do Norte (Geofencing).
- **[RF-05]** O formulário de criação de locais deve capturar as coordenadas (latitude e longitude) automaticamente ao clicar no mapa interativo.
- **[RF-06]** O sistema deve permitir a criação direta de Roteiros (Itineraries) por usuários autenticados, compondo-os com locais previamente cadastrados.
- **[RF-07]** A listagem de roteiros deve aplicar filtros: Turistas veem apenas os próprios roteiros, e os Guias/Admins visualizam todos.
- **[RF-08]** O sistema deve registrar avaliações (Reviews) atribuindo uma nota de 1 a 5 e um comentário textual.
- **[RF-09]** Visitantes sem autenticação devem conseguir explorar o mapa de locais e visualizar avaliações.

Incompleto/Pendente:

- **[RF-10]** O sistema deve implementar de forma completa a máquina de estados para roteiros (Rascunho → Em Análise → Publicado), pois atualmente existe apenas um CRUD de criação direta.
- **[RF-11]** O sistema deve suportar o upload de imagens reais (multipart/form-data) para Pontos Turísticos e Avaliações, substituindo a interface visual que se encontra como "Coming Soon".
- **[RF-12]** A tela de detalhes do roteiro deve consumir e exibir os dados da API externa de previsão do tempo (OpenWeatherMap), substituindo os mocks atuais da interface.
- **[RF-13]** O sistema deve permitir exportar um roteiro no formato PDF.
- **[RF-14]** O painel do administrador deve exibir gráficos estatísticos avançados sobre os acessos.

Requisitos Não Funcionais (RNF)

Já Implementado:

- **[RNF-01]** A aplicação deve ser estruturada em uma arquitetura *Monorepo* gerenciada por workspaces.
- **[RNF-02]** O back-end deve ser modularizado com o framework NestJS.
- **[RNF-03]** A persistência de dados deve ocorrer num banco PostgreSQL manipulado pelo Prisma ORM.
- **[RNF-04]** A interface deve ser uma Single Page Application (SPA) construída em Vue 3 (Composition API), Vite, Tailwind CSS e Pinia.
- **[RNF-05]** O armazenamento de senhas deve ser feito exclusivamente de forma criptografada (hash unidirecional).
- **[RNF-06]** O acesso a rotas sensíveis no back-end deve ser protegido por *Guards* no NestJS.
- **[RNF-07]** Os mapas interativos devem ser gerados de forma gratuita com a ferramenta *open source* Leaflet.js.
- **[RNF-08]** O ambiente de desenvolvimento da aplicação e do banco deve estar configurado em contêineres Docker e Docker Compose.
- **[RNF-09]** A API REST deve ter documentação viva e interativa disponível no Swagger/OpenAPI.

Incompleto/Pendente:

- **[RNF-10]** O interceptor HTTP responsável pelo *Refresh Token* deve ser corrigido definitivamente para evitar loops infinitos ou falhas nas filas de requisição (erro 401).
- **[RNF-11]** A aplicação deve implementar Testes Automatizados E2E (utilizando ferramentas como Cypress ou Playwright) para validar os fluxos completos sem intervenção humana.
- **[RNF-12]** A infraestrutura de *deploy* contínuo (Vercel/Render) deve ser estabilizada, solucionando os conflitos entre as engines e versões do Node.js (v20 vs v24).
- **[RNF-13]** O front-end e o UX/UI precisam ser polidos com *loading states* (estados de carregamento) e tratamentos amigáveis de mensagens de erro (*toasts*).
- **[RNF-14]** O banco de dados necessita de revisão nas tabelas intermediárias do Prisma para assegurar o funcionamento da exclusão em cascata (*cascading deletes*).

- **[RNF-15]** O código deve passar por uma varredura rigorosa para aplicação de *Clean Code*, removendo variáveis inativas e garantindo a tipagem dos retornos das APIs no TypeScript.